

Design Patterns: Decorator, Observer

11.1 Decorator Design Pattern

Design Pattern-ul Decorator specifica un mod dinamic de adaugare de functionalitati unor obiecte existente fara a modifica pe acesta.

Este un design pattern structural.

Caracteristici

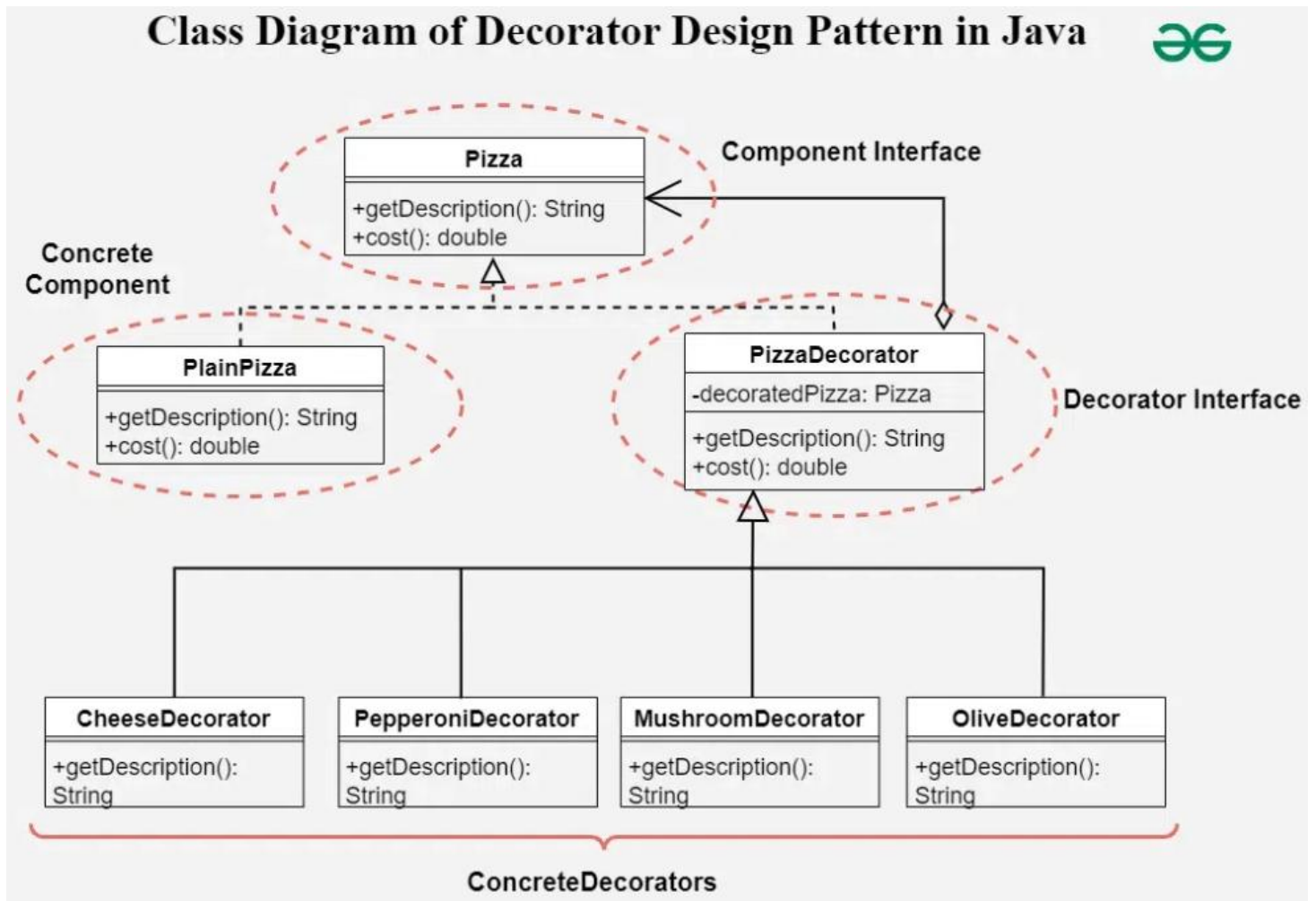
- ofera flexibilitate si extensibilitate, permitand compunerea de obiecte cu diverse comportamente la runtime
- respecta principiul Open/Closed, prin faptul ca pot fi adaugati decoratori fara modificarea codului existent
- este utilizat in situatii in care o varietate de facilitati optionale trebuiesc adaugate obiectelor intr-o maniera flexibila si reutilizabila. Ex. formatare de text, interfete grafice, personalizare de produse

Componente

- **Component Interface** – interfata/clasa abstracta care defineste functionalitatile centrale. Este clasa de baza pentru componentele concrete dar si pentru decorator
- **Concrete component** – clasa care implementeaza *Component Interface* si ofera comportamentul de baza
- **Decorator** – o clasa abstracta care implementeaza de asemenea *Component Interface* si are referinta la un obiect/instanta *Component* (*Concrete component*).
- **Concrete Decorator** – clase care mostenesc *Decorator* si adauga functionalitati aditionale instantei *Component* (*Concrete component*)

Laborator 11

Exemplu:



Codul aferent:

1. Component interface

```
// Pizza.java
public interface Pizza {
    String getDescription();
    double cost();
}
```

2. Concrete Component

```
// PlainPizza.java
public class PlainPizza implements Pizza {
    @Override
    public String getDescription() {
        return "Plain pizza";
    }

    @Override
```

Laborator 11

```

    public double cost() {
        return 8.0; // Base price of the pizza
    }
}

```

3. Abstract Decorator

```

// PizzaDecorator.java
public abstract class PizzaDecorator implements Pizza {
    protected Pizza decoratedPizza;

    public PizzaDecorator(Pizza decoratedPizza) {
        this.decoratedPizza = decoratedPizza;
    }

    @Override
    public String getDescription() {
        return decoratedPizza.getDescription();
    }

    @Override
    public double cost() {
        return decoratedPizza.cost();
    }
}

```

4. Concrete Decorators

```

// CheeseDecorator.java
public class CheeseDecorator extends PizzaDecorator {
    public CheeseDecorator(Pizza decoratedPizza) {
        super(decoratedPizza);
    }

    @Override
    public String getDescription() {
        return decoratedPizza.getDescription() + ", cheese";
    }

    @Override
    public double cost() {
        return decoratedPizza.cost() + 1.5; // Cost of cheese topping
    }
}

// PepperoniDecorator.java
public class PepperoniDecorator extends PizzaDecorator {
    public PepperoniDecorator(Pizza decoratedPizza) {
        super(decoratedPizza);
    }

    @Override
    public String getDescription() {
        return decoratedPizza.getDescription() + ", pepperoni";
    }
}

```

Laborator 11

```

@Override
public double cost() {
    return decoratedPizza.cost() + 2.0; // Cost of pepperoni topping
}

// MushroomDecorator.java
public class MushroomDecorator extends PizzaDecorator {
    public MushroomDecorator(Pizza decoratedPizza) {
        super(decoratedPizza);
    }

    @Override
    public String getDescription() {
        return decoratedPizza.getDescription() + ", mushrooms";
    }

    @Override
    public double cost() {
        return decoratedPizza.cost() + 1.0; // Cost of mushroom topping
    }
}

// OliveDecorator.java
public class OliveDecorator extends PizzaDecorator {
    public OliveDecorator(Pizza decoratedPizza) {
        super(decoratedPizza);
    }

    @Override
    public String getDescription() {
        return decoratedPizza.getDescription() + ", olives";
    }

    @Override
    public double cost() {
        return decoratedPizza.cost() + 0.75; // Cost of olive topping
    }
}

// PizzaShop.java
public class PizzaShop {
    public static void main(String[] args) {
        Pizza pizza = new PlainPizza();
        System.out.println(pizza.getDescription() + " $" + pizza.cost());

        pizza = new CheeseDecorator(pizza);
        System.out.println(pizza.getDescription() + " $" + pizza.cost());

        pizza = new PepperoniDecorator(pizza);
        System.out.println(pizza.getDescription() + " $" + pizza.cost());

        pizza = new MushroomDecorator(pizza);
        System.out.println(pizza.getDescription() + " $" + pizza.cost());

        pizza = new OliveDecorator(pizza);
        System.out.println(pizza.getDescription() + " $" + pizza.cost());
    }
}

```

Laborator 11

```
}
}
```

Component Interface (Pizza): definește metodele `getDescription()` și `cost()` pe care toate instanțele pizza și decorator trebuie să le implementeze.

Concrete Component (PlainPizza): Implementează interfata Pizza și reprezintă o pizza de bază cu 'description' și 'cost'.

Abstract Decorator (PizzaDecorator): Implementează interfata Pizza și conține referința la obiectul Pizza. Această clasă redirectionează apelurile metodelor către obiectul pizza și poate fi extinsă de către decoratori.

Decoratori concreți (CheeseDecorator, PepperoniDecorator, MushroomDecorator, OliveDecorator): mostenesc PizzaDecorator și adaugă ingrediente specifice. Fiecare decorator modifică 'description' și 'cost' al pizzei curente prin adăugare de noi ingrediente.

Codul Client (PizzaShop): demonstrează cum se poate crea o instanță PlainPizza și apoi decora cu diverse ingrediente. Ingredientele se adaugă dinamic iar descrierea (description) și costul (cost) sunt actualizate corespunzător.

11.2 Observer Design Pattern

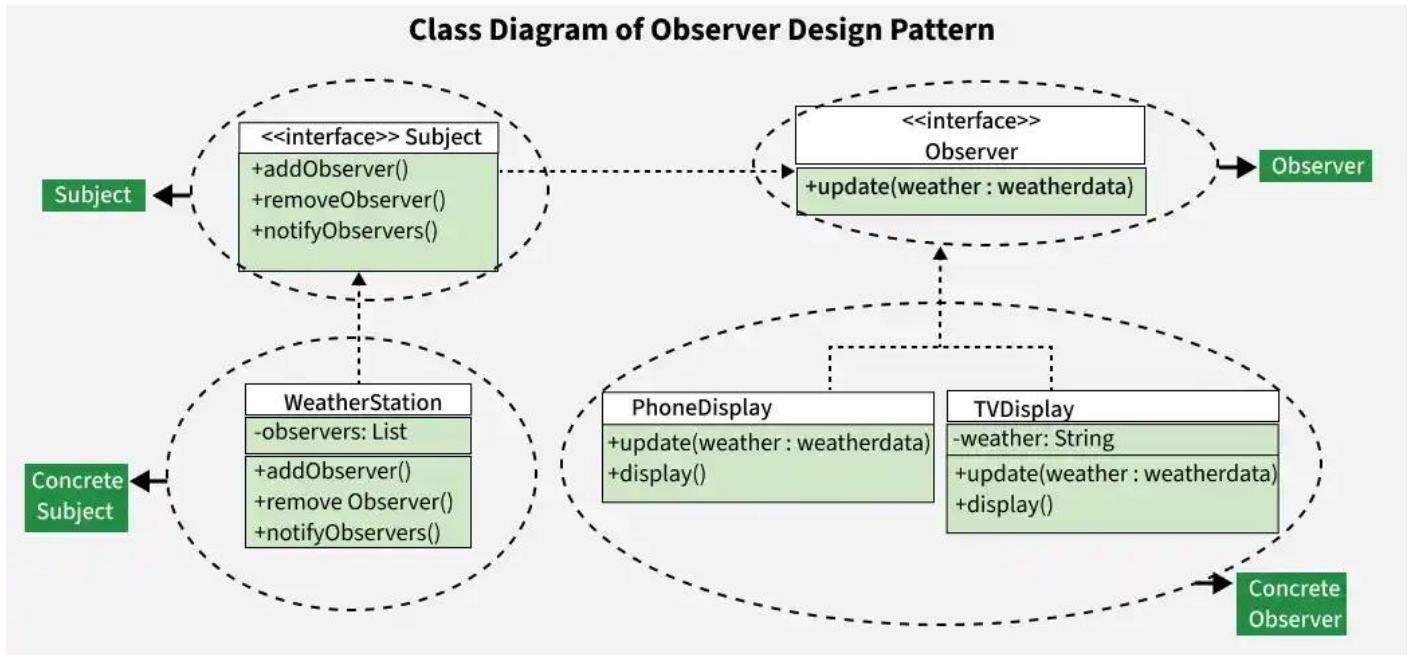
Design Pattern-ul *Observer* definește o relație de dependență 1 la n între obiecte astfel încât când un obiect își schimbă starea, toți dependenții lui sunt notificați și actualizați automat. Folosirea acestui pattern implică existența unui obiect cu rolul de *subject*, care are asociată o listă de obiecte dependente, cu rolul de *observers*, pe care le apelează automat de fiecare dată când se întâmplă o acțiune.

Acest pattern este de tip *Behavioral* (comportamental), deoarece facilitează o organizare mai bună a comunicărilor dintre clase în funcție de rolurile/comportamentul acestora.

Observer se folosește în cazul în care mai multe clase (*observers*) depind de comportamentul unei alte clase (*subject*), în situații de tipul:

- o clasă implementează/reprezintă logica, componenta de bază, iar alte clase doar folosesc rezultate ale acesteia (monitorizare).
- o clasă efectuează acțiuni care apoi pot fi reprezentate în mai multe feluri de către alte clase (view-uri ca în figura de mai jos).

Laborator 11

**Subiect (observabilul)**

- nu trebuie sa stie ce fac observatorii, trebuie doar sa mentina referinte catre obiecte de acest tip
- nu este interesat ce fac observatorii cu datele
- ofera o modalitate de a mentine o lista de *Observatori*, si de a o modifica la runtime
- cand apar modificari (ex. se schimba starea sa, valorile unor variabile etc) atunci notifica toti observatorii

Observatorul

- defineste o interfata a carei metoda/metode sunt definite pentru a se asigura ca toti observatorii primesc actualizarile necesare
- implementare:
 - o toti observatorii pentru un anumit subiect trebuie sa implementeze aceasta interfata
 - o ofera una sau mai multe metode care sa poata fi invocate de catre *Subiect* pentru a notifica o schimbare. Ca argumente se poate primi chiar instanta subiectului sau obiecte speciale care reprezinta evenimentul ce a provocat schimbarea.

ConcreteSubject (clasa instantiabila subiect)

- este o clasa concreta (instantiabila) care pe langa comportamentele necesare, are si abilitatea de a se comporta ca un Subiect (interfata Subiect/Observable)
- orice modificare a datelor interne, notifica observatorii inregistrati

ConcreteObserver (clasa instantiabila observer)

- se comporta ca observator, si reactioneaza (asculta) la notificarile despre modificarile in subiect/observabil

Utilizat frecvent in:

Laborator 11

- gestionarea unor evenimente (ex. mouse, tastatura). De ex in interfete UI
- mentinerea consistentei intre doua obiecte relationate, fara implementare “tight coupling”
- implementare de mecanisme publish-subscribe

Exemplu de notificare a doua tipuri de investitori la bursa:

```
// fisierul StockInterface.java
package stockdemo;
public interface StockInterface {
    public void register(Investor investor);
    public void unregister(Investor investor);
    public void notifyChange();
}

// fisierul Investor.java
package stockdemo;
public interface Investor {
    public void update(Stock stock);
}
```

Laborator 11

```
// fisierul Investor_A.java
package stockdemo;
public class Investor_A implements Investor {
    @Override
    public void update(Stock stock) {
        System.out.println("InvestorA: Price of " + stock.name + " has changed to " +
stock.getPrice());
    }
}

// fisierul Investor_B.java
package stockdemo;
public class Investor_B implements Investor {
    @Override
    public void update(Stock stock) {
        System.out.println("InvestorB: Price of " + stock.name + " has changed to " +
stock.getPrice());
    }
}

// fisierul Stock.java
package stockdemo;
import java.util.ArrayList;
public class Stock implements StockInterface{
    private double price;
    String name;
    private ArrayList<Investor> observers;
    public Stock(String name , double price)
    {
        this.name=name;
        this.setPrice(price);
        observers=new ArrayList<Investor>();
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }
    @Override
    public void register(Investor investor){
        if(!observers.contains(investor))
        {
            observers.add(investor);
        }
        else {
            System.out.println("Investor already registered");
        }
    }
    @Override
    public void unregister(Investor investor){
        if(observers.contains(investor))
        {
            observers.remove(investor);
        }
    }
}
```


Laborator 11

```

        else {
            System.out.println("Investor not registered");
        }
    }
    @Override
    public void notifyChange() {
        for (Investor investor : observers) {
            investor.update(this);
        }
    }
}

```

11.3 Probleme

11.3.1 In proiectul ProiectareSoftware adaugati sursele din fisierul **Laborator11_starter.zip**.

Implementati Design Pattern-ul Observable in acest proiect. Faceti ca YouTubeChannel sa fie observat de instantele MediaInterested.

- implementati Observer in clasa MediaInterested
- implementati Subject in clasa YouTubeChannel
- notificati observatorii in metoda uploadVideo
- adaugati/eliminati observatori inainte de apelul uploadVideo() in main() astfel incat primul apel sa gaseasca 2 observatori inregistrati, iar al doilea sa gaseasca doar unul.

Adaugati sursele in GitHub.

11.3.2 In proiectul Students, implementati Design Patternul Decorator pentru a masura timpul de executie al fiecarei instante de export. Nu modificati functia de export.

Utilizati **System.currentTimeMillis()** pentru a obtine timpul in milisecunde in decorator. Tot in decorator afisati in consola acest timp.

Adaugati sursele in GitHub

11.4 Referinte

Decorator

<https://refactoring.guru/design-patterns/decorator>

Decorator Pattern in Java

<https://www.baeldung.com/java-decorator-pattern>

Decorator Method Design Pattern in Java

<https://www.geeksforgeeks.org/java/decorator-design-pattern-in-java-with-example/>

Observer in Java

<https://refactoring.guru/design-patterns/observer/java/example>

The Observer Pattern in Java

<https://www.baeldung.com/java-observer-pattern>

Observer Design Pattern

<https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/>

Java System currentTimeMillis() Metod

https://www.tutorialspoint.com/java/lang/system_currenttimemillis.htm

Laborator 11

Hints:

11.3.1

```
public class MediaInterested implements Observer { 20
```

```
public class YouTubeChannel implements Subject { no usages new *  
    private List<Observer> observers = new ArrayList<>(); 3 usages  
    private String channelName; 3 usages
```

```
public interface Subject { no usages new *  
    void addObserver(Observer observer); no usages new *  
    void removeObserver(Observer observer); no usages new *  
    void notifyObservers(String message); no usages new *  
}
```

```
public interface Observer { no usages  
    void update(String message);  
}
```

Laborator 11

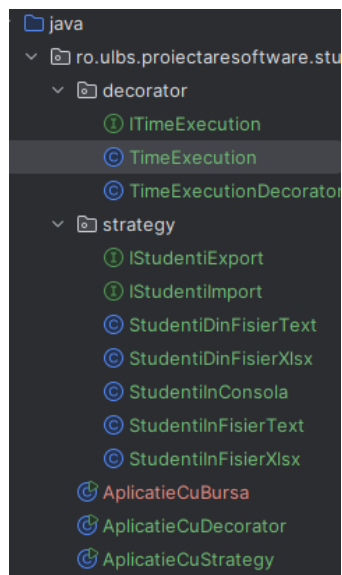
11.3.2

```

public class AplicatieCuDecorator { new *
    static void main(String[] args) { new *
        List<Student> studentiCuNote = Arrays.asList(
            new Student( numarulMatricol: 1025, prenume: "Andrei", nume: "Popa", formatieDeStudiu: "ISM141/2", nota: 8.70),
            new Student( numarulMatricol: 1024, prenume: "Ioan", nume: "Mihalcea", formatieDeStudiu: "ISM141/1", nota: 10),
            new Student( numarulMatricol: 1026, prenume: "Anamaria", nume: "Prodan", formatieDeStudiu: "TI131/1", nota: 8.90),
            new Student( numarulMatricol: 1029, prenume: "Bianca", nume: "Popescu", formatieDeStudiu: "TI131/1", nota: 10),
            new Student( numarulMatricol: 1029, prenume: "Maria", nume: "Pana", formatieDeStudiu: "TI131/2", nota: 4.10),
            new Student( numarulMatricol: 1029, prenume: "Gabriela", nume: "Mohanu", formatieDeStudiu: "TI131/2", nota: 7.33),
            new Student( numarulMatricol: 1029, prenume: "Marius", nume: "Nasta", formatieDeStudiu: "TI131/2", nota: 3.20),
            new Student( numarulMatricol: 1029, prenume: "Marius", nume: "Nasta", formatieDeStudiu: "TI131/1", nota: 5.12),
            new Student( numarulMatricol: 1029, prenume: "Andrei", nume: "Dobrescu", formatieDeStudiu: "TI131/2", nota: 2.22)
        );
        List<IStudentiExport> strategies = Arrays.asList(
            new StudentiInConsola(),
            new StudentiInFisierText( fileName: "studentiStrategyText.txt"),
            new StudentiInFisierXlsx( fileName: "studentiStrategyExcel.xlsx")
        );

        for (IStudentiExport strategy : strategies) {
            TimeExecutionDecorator decorator = new TimeExecutionDecorator(strategy, studentiCuNote);
            long time = decorator.executionTime();
            System.out.println("Execution time: " + time + " for " + strategy);
        }
    }
}

```



```

public class TimeExecutionDecorator extends TimeExecution {
    private List<Student> studenti; 2 usages

    public TimeExecutionDecorator(IStudentiExport exporter, List<Student> studenti) {
        super(exporter);
        this.studenti = studenti;
    }

    public long executionTime() { 1 usage new *
        long execTime = super.executionTime(studenti);
        return execTime;
    }
}

```

```

public interface ITimeExecution { 1 usage 2 implements
    long executionTime(List<Student> studenti);
}

```